

Amaze Dex

Project Report

Luv Bharat Jain Om Sontakke

GitHub - <https://github.com/OMSONTAKKE003/AmazeDex/tree/main>

Acknowledgements

We would like to express our heartfelt gratitude to our mentors and guides, Arhan Chavre and Sahil Apage for their invaluable support and encouragement throughout the course of this project. We are also thankful to the Society of Robotics and Automation, VJTI, which provided us the platform and resources required to realize our vision with “AmazeDex”.

Index

SR. NO.	TOPIC	PAGE NO.
1	Overview	4
2	Forms of Dextrous Manipulation	5
3	Related Work	5
4	Simulation	6
5	Hardware	7
6	Controls	8
7	6D pose estimation in real	9
8	Sim 2 Real	10
9	Training	12
10	Flow Charts	16
11	Comparison Charts	17
12	Graphs	18
13	Weekly Progress	20
14	Resources and References	22

Overview

The project “AmazeDex” aims to train a multi fingered robotic hand to autonomously rotate a cube to any desired target orientation using deep reinforcement learning. The platform utilizes the Amazing Hand by Pollen Robotics.

The robot receives information about the object’s current orientation and target orientation and learns finger movements that gradually align the object with the target pose. The training is performed entirely in a simulated environment, where the agent learns through trial and error by maximizing rewards based on orientation accuracy and grasp stability. The project demonstrates the application of deep reinforcement learning for dexterous manipulation and highlights the challenges of controlling multi-finger robotic systems for complex in hand manipulation and deploying it on hardware

This project is a union of mechanical design, computer vision, and reinforcement learning to achieve dexterity. AmazeDex shows the creative side of robotics, where mechanics and human-like movements come together. The aim is not just to make the hand move, but to make it perform the task properly, efficiently, and with smooth movements.

Forms of Dextrous Manipulation

We mainly tried 2 main forms of dextrous manipulation -

1. Lift the cube up and then rotate the cube about the axis on which the cube was spawned
2. Rotate the cube to target face taking target face as an input from the user

In approach one the main issue was reward hacking , when then fingers grasped the cube , instead of rotation the hand setup became idle . The hand was unable to lift the cube off the base of the palm.

The second approach gave decent results. The hand started continuously to rotate the cube to target face . We achieved a maximum of 34 percent success rate and stable convergent success rate of about 28 percent in simulation .

Related Work

Dexterous manipulation with robotic hands has been a major research focus for over two decades. The Shadow Dexterous Hand, from Shadow Robot Company, is one of the most established platforms, reproducing human hand kinematics with 24 degrees of freedom, 20 of which are independently controllable. It integrates over 100 sensors running at up to 1kHz, making it a strong research benchmark, though its tendon-driven actuation is known to reduce long-term reliability compared to direct-drive designs.

Newer platforms address this with direct-drive actuation instead. The Wuji Hand offers 20 degrees of freedom with four independently driven joints per finger, using direct-drive rotary joints, and rates at least 300,000 grasp cycles with ± 1 mm positioning repeatability. This favors mechanical robustness and easier integration over the Shadow Hand's higher sensor density. The Amazing Hand, built with low-cost Feetech servos, follows a similar philosophy at a more accessible price point, trading sensing complexity for affordability and simplicity.

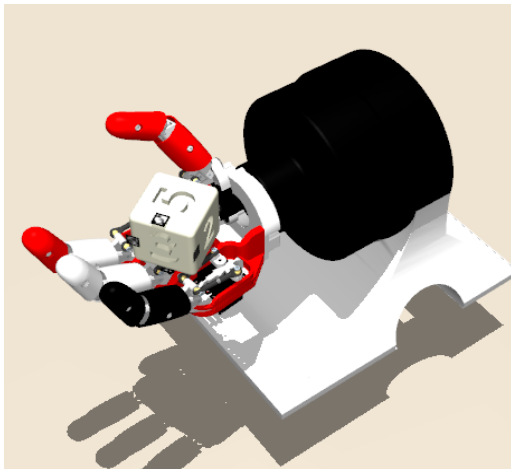
On the control side, OpenAI's work on in-hand manipulation showed that reinforcement learning policies trained mostly in simulation could transfer to real hardware, enabling a Shadow Hand to reorient objects and later solve a Rubik's cube one-handed. This proved that large-scale simulated training could produce robust, near-human manipulation without hand-coded control logic, an approach that has since influenced sim-to-real research on other platforms, including retargeting studies on the Wuji Hand. Together, this work outlines where our project fits: a simple, servo-driven hand currently using scripted control, with learned control policies as a possible future direction.

Simulation -

The Amazing Hand by Pollen Robotics is a four-fingered hand with 8 degrees of freedom (DOF). The hand was simulated and trained in MuJoCo, which was chosen because it is lightweight, easy to work with, and provides a simple way to represent real-world physical parameters.

To simulate the hand, two separate XML files were created in MuJoCo. The “robot.xml” file contains the hand’s structure, including its joints, actuators, control ranges, forces, colours, and other physical properties. The “scene.xml” file contains the simulation environment, including the ground, a custom camera, lighting, and the cube used for manipulation. All relevant collisions , physical parameters were added and verified to ensure that the simulation behaved correctly.

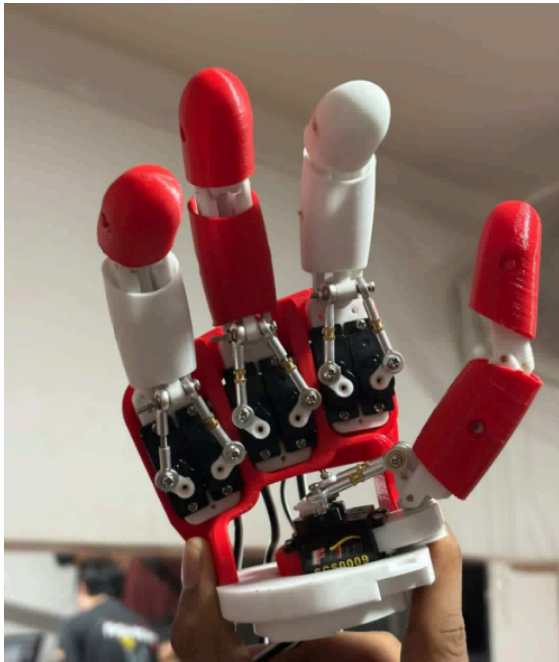
[Pollen_robotics](#)



Since no actual stand was available in open source to hold the hand fixed in horizontal position , we designed a mechanical stand to hold the hand.

Hardware

The hand is a 3D-printed replica of Pollen Robotics' Amazing hand which was constructed entirely from PLA filament. The fingers were assembled using standard dowel pins, while toothpicks provided a practical, cost-effective solution for attaching the finger shells. Standard hardware secures all the joints, and the completed hand is mounted on a two-piece printed stand.



While using the hardware we came across many unexpected problems -

1. Gimbal of one of the fingers snapped very often , we tried printing it on different settings and filaments but still it broke often
2. Waveshare bus servo adaptor died with no obvious cause - our best guess to why adaptors died is - “Inductive Back-EMF SurgeThe Mechanism: When a servo suddenly stops, reverses, or hits a mechanical jam, its internal motor windings release stored magnetic energy.The Result: This sends a massive, high-voltage spike backward into the adapter board's power rail”
3. Speed control of servos , in order to prevent breaking the movements of our hardware, are slower than in simulation.

Controls

1. Amazing hand GUI

This is a ready-made open-source toolkit on GitHub that gives the Amazing Hand a proper GUI instead of a raw script. It opens up a window with per-finger sliders, open/close buttons, speed settings, and keyboard shortcuts, plus live charts for position, load, temperature, and voltage. We can also save custom hand poses and chain them into sequences (like a wave or finger roll) that get stored in a config file and replayed anytime.

[GUI](#)

2. Feetech Software

This is Feetech's own free tool for setting up their servos. We used it to scan for connected servos, change IDs, set baud rate, move servos manually with sliders, and check things like voltage and temperature. We use it as a first check, just to make sure each servo powers on and responds correctly before writing any code against it.

[Feetech_software](#)

3. Rustypot

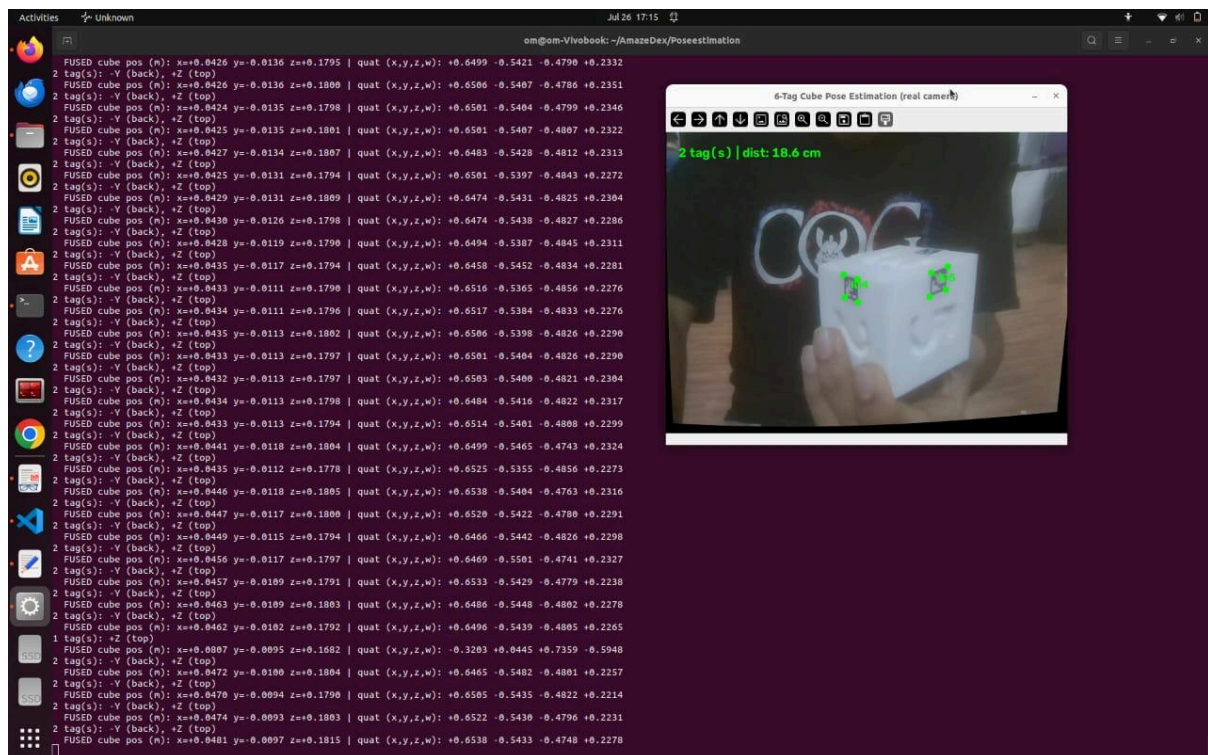
Rustypot is the library our script actually uses to talk to the servos. It's built in Rust but works from Python, and gives us simple commands like "enable torque," "move to this position," and "read current position." It comes from Pollen Robotics' open-source tools. Basically: Feetech's software is for manual testing, Rustypot is what lets us control the servos automatically in code.

6D Pose estimation in real

In simulation, we could directly get the cube's quaternion from MuJoCo's ground truth, but this information is not available in real life. This became one of the main challenges we faced during the sim-to-real transfer.

To solve this, we explored different 6D pose estimation approaches and searched through several GitHub repositories, including NVIDIA's DOPE (Deep Object Pose) and MegaPose6D. We also looked into classical OpenCV methods and tried using a PnP solver. However, most of these approaches could struggle with occlusion and changes in lighting. Because of these limitations, we decided to move forward with AprilTags, which provided a simpler and more reliable way to estimate the cube's pose in our setup.

[AprilTags](#)



Sim 2 Real

We tried our best to transfer the trained policy from simulation to the real Amazing Hand, but we faced several practical issues. Our simulation and real setup could not become perfect digital twins, so the same action did not always produce the same movement.

Major issues we faced:

- **Origin and coordinate mismatch:** The joint origins and coordinate frames in simulation were not exactly the same as on the real hand. This caused differences in finger positions and movements.
- **Servo calibration:** The real servos had calibration errors, and different joints showed different amounts of error. Because of this, the joint positions in the real hand did not always match the positions expected by the policy.
- **Difference in joint movement:** The fingers moved differently in simulation and on the real hand. Factors like servo speed, delay, backlash and mechanical friction affected the real movement.
- **Simulated vs real physics:** Friction, mass, damping and contact behaviour were difficult to match exactly. Even after tuning these parameters, the cube behaved differently in the real setup.
- **Domain randomization:** We tried randomizing parameters such as friction, mass and actuator behaviour to make the policy more robust. However, randomization could not cover all the differences present in the actual hardware.
- **Camera calibration:** The camera position and viewpoint in the real setup were different from the simulated setup. This affected the cube's detected position and orientation.

Because of these combined issues, the policy that worked in simulation but could not reliably reproduce the same behaviour on the real hand.



Training

1. PPO (Proximal Policy Optimization)

PPO was initially used because it provides stable policy updates and is widely suited to continuous-control problems. It learns an actor policy while using a critic to estimate the value of states. PPO performed reasonably well for basic manipulation, but learning became difficult as the cube-rotation task required more precise and long-horizon coordination.

Our first approach was PPO as it was most widely used in major dextrous manipulation papers and research papers. The issue faced was that it was very slow, the policy had to be trained for millions of steps before a mistake in code was realised.

PPO was prone to reward hacking and difficult to tune hyperparameters

Algorithm 1 PPO-Clip

- 1: Input: initial policy parameters θ_0 , initial value function parameters ϕ_0
- 2: **for** $k = 0, 1, 2, \dots$ **do**
- 3: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
- 4: Compute rewards-to-go $\hat{R}_t$.
- 5: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k} .
- 6: Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \min \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.

- 7: Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg \min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \left(V_{\phi}(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.

- 8: **end for**
-

2. PPO + Curriculum Learning

Curriculum learning was introduced to gradually increase task difficulty, allowing the policy to first learn simpler rotations or easier initial configurations before attempting the complete task. This improved learning in the early stages, the hand learned to grasp the cube but this was the very issue that led us to drop this algorithm. After

various iterations we found that the model was always reward hacking, it learnt to grasp the cube but did not try to rotate it .

Hence we decided to drop it and move to the next algorithm

3. DDPG + HER (Hindsight Experience Replay)

HER was combined with DDPG to improve learning from unsuccessful attempts. Instead of treating a failed trajectory as completely unsuccessful, HER relabels the trajectory using an achieved goal, allowing the agent to learn useful information from failed manipulation attempts. This helped address sparse-goal learning, but the combination still struggled with the complexity and instability of dexterous contact interactions.

Algorithm 1 Deep Deterministic Policy Gradient

```

1: Input: initial policy parameters  $\theta$ , Q-function parameters  $\phi$ , empty replay buffer  $\mathcal{D}$ 
2: Set target parameters equal to main parameters  $\theta_{\text{targ}} \leftarrow \theta$ ,  $\phi_{\text{targ}} \leftarrow \phi$ 
3: repeat
4:   Observe state  $s$  and select action  $a = \text{clip}(\mu_{\theta}(s) + \epsilon, a_{\text{Low}}, a_{\text{High}})$ , where  $\epsilon \sim \mathcal{N}$ 
5:   Execute  $a$  in the environment
6:   Observe next state  $s'$ , reward  $r$ , and done signal  $d$  to indicate whether  $s'$  is terminal
7:   Store  $(s, a, r, s', d)$  in replay buffer  $\mathcal{D}$ 
8:   If  $s'$  is terminal, reset environment state.
9:   if it's time to update then
10:    for however many updates do
11:      Randomly sample a batch of transitions,  $B = \{(s, a, r, s', d)\}$  from  $\mathcal{D}$ 
12:      Compute targets

$$y(r, s', d) = r + \gamma(1 - d)Q_{\phi_{\text{targ}}}(s', \mu_{\theta_{\text{targ}}}(s'))$$

13:      Update Q-function by one step of gradient descent using

$$\nabla_{\phi} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} (Q_{\phi}(s, a) - y(r, s', d))^2$$

14:      Update policy by one step of gradient ascent using

$$\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} Q_{\phi}(s, \mu_{\theta}(s))$$

15:      Update target networks with

$$\begin{aligned} \phi_{\text{targ}} &\leftarrow \rho \phi_{\text{targ}} + (1 - \rho) \phi \\ \theta_{\text{targ}} &\leftarrow \rho \theta_{\text{targ}} + (1 - \rho) \theta \end{aligned}$$

16:    end for
17:  end if
18: until convergence

```

4. SAC (Soft Actor-Critic)

SAC was then tested because it combines off-policy learning, experience replay, entropy-based exploration, and a stochastic policy. Unlike DDPG, SAC maintains stronger exploration while learning a value function from replayed experience. This was particularly useful for discovering different finger coordination and contact strategies needed for cube rotation.

Algorithm 1 Soft Actor-Critic

- 1: Input: initial policy parameters θ , Q-function parameters ϕ_1, ϕ_2 , empty replay buffer $\mathcal{D}$
- 2: Set target parameters equal to main parameters $\phi_{\text{targ},1} \leftarrow \phi_1, \phi_{\text{targ},2} \leftarrow \phi_2$
- 3: **repeat**
- 4: Observe state s and select action $a \sim \pi_\theta(\cdot|s)$
- 5: Execute a in the environment
- 6: Observe next state s' , reward r , and done signal d to indicate whether s' is terminal
- 7: Store (s, a, r, s', d) in replay buffer $\mathcal{D}$
- 8: If s' is terminal, reset environment state.
- 9: **if** it's time to update **then**
- 10: **for** j in range(however many updates) **do**
- 11: Randomly sample a batch of transitions, $B = \{(s, a, r, s', d)\}$ from $\mathcal{D}$
- 12: Compute targets for the Q functions:

$$y(r, s', d) = r + \gamma(1 - d) \left(\min_{i=1,2} Q_{\phi_{\text{targ},i}}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}'|s') \right), \quad \tilde{a}' \sim \pi_\theta(\cdot|s')$$

- 13: Update Q-functions by one step of gradient descent using

$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} (Q_{\phi_i}(s, a) - y(r, s', d))^2 \quad \text{for } i = 1, 2$$

- 14: Update policy by one step of gradient ascent using

$$\nabla_\theta \frac{1}{|B|} \sum_{s \in B} \left(\min_{i=1,2} Q_{\phi_i}(s, \tilde{a}_\theta(s)) - \alpha \log \pi_\theta(\tilde{a}_\theta(s)|s) \right),$$

where $\tilde{a}_\theta(s)$ is a sample from $\pi_\theta(\cdot|s)$ which is differentiable wrt θ via the reparametrization trick.

- 15: Update target networks with

$$\phi_{\text{targ},i} \leftarrow \rho \phi_{\text{targ},i} + (1 - \rho) \phi_i \quad \text{for } i = 1, 2$$

- 16: **end for**
 - 17: **end if**
 - 18: **until** convergence
-

Why SAC Was the most Successful

SAC produced the strongest overall performance because its characteristics matched the requirements of the task particularly well. Cube manipulation involves continuous actions, sparse success events, nonlinear contact dynamics, and many possible ways of achieving the same rotation. SAC's entropy-driven exploration encouraged the hand to explore different finger coordination strategies instead of prematurely committing to one behavior. Its off-policy replay buffer also allowed useful experiences to be reused many times, making it more sample-efficient than the on-policy PPO approach.

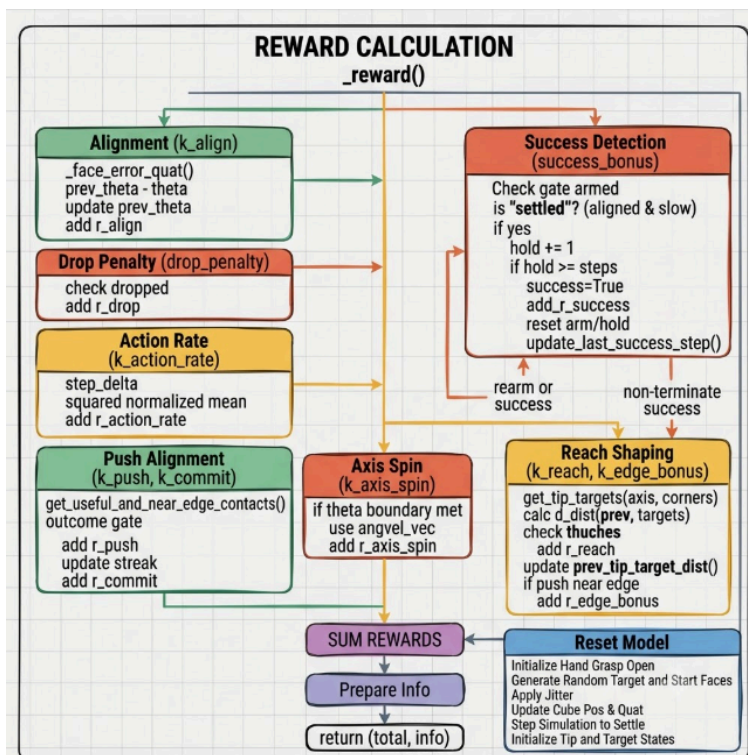
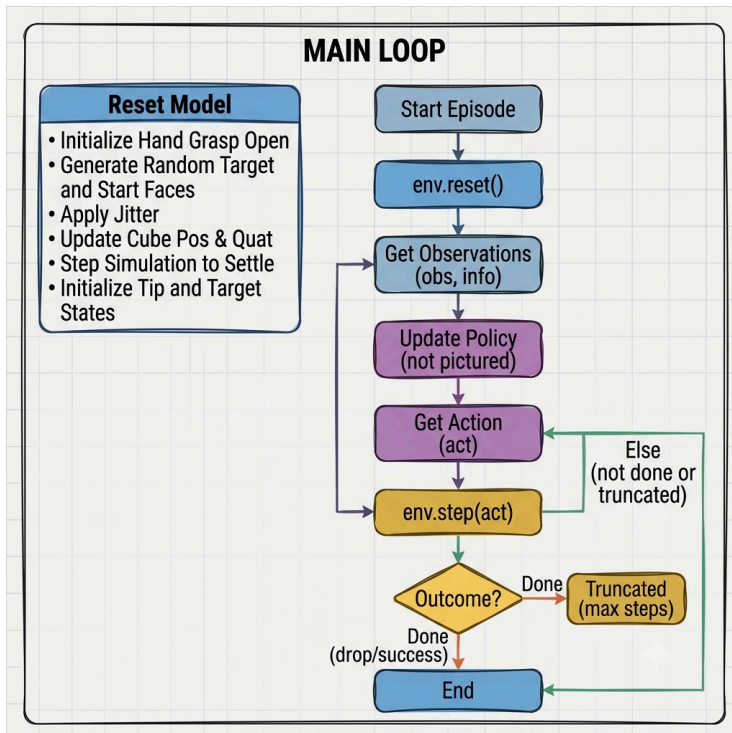
Compared with DDPG, SAC's stochastic exploration made learning more robust to the noisy and highly nonlinear reward landscape. Curriculum learning could help with exploration, but SAC was ultimately able to learn the complete manipulation task without relying as heavily on carefully designed difficulty transitions.

SAC being off policy learnt the actions to effectively rotate the cube and do manipulation in a shorter duration. It in comparison to PPO provided drastically better results from around 150k timesteps. At just 2 million timesteps our model reached accuracy of about 80 percent .

Changes that made a major improvement

1. Reducing the size of the cube (48 mm) , using a slight fillet (fillet radius 4 mm) on the edges of the cube, reducing the mass and changing initial spawn position.
2. Increasing the push reward from 0.15 to 0.50
3. Increasing the reach reward from 0.05 to 0.10
4. Reweighting success bonus and drop rate . Success rate from 26 to 16 and drop rate from 5.1 to 2.0 .
5. Changing the reset model and start model. If we randomise both start face and target face both, the model has to learn 36 different combinations to avoid this we increased timesteps and fixed start to one face by doing this we reduced the combinations possible .
6. After training without randomisation , we randomised one metric at a time and randomised terms such as mass of cube , friction , camera latency etc.

Flow Chart

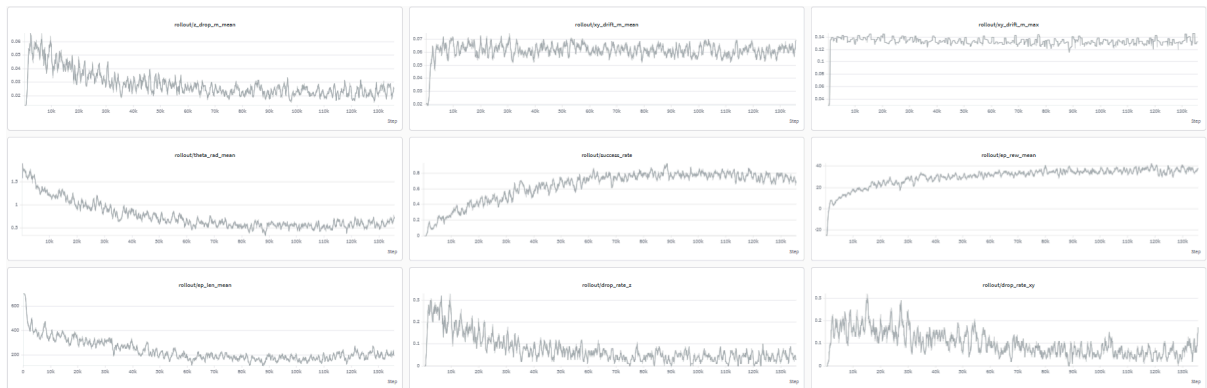
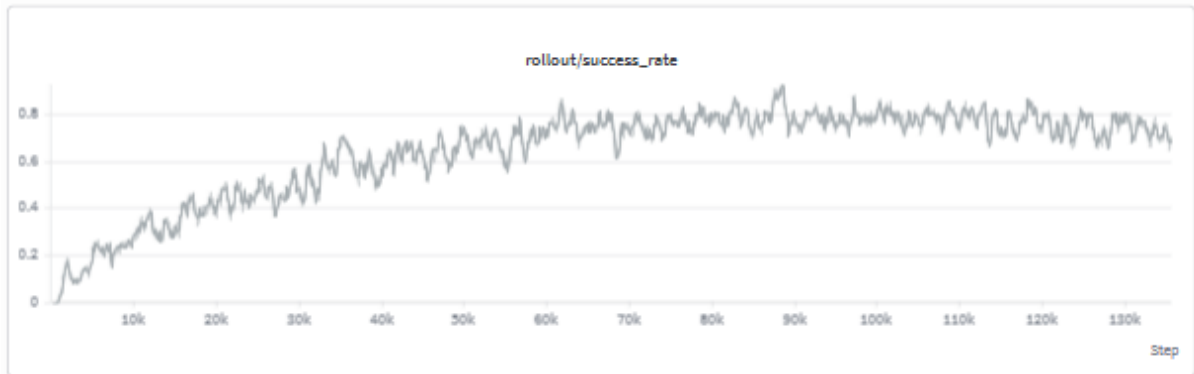


Comparison Charts

Parameter	PPO	PPO Curriculum +	SAC	DDPG + HER	SAC (domain randomised)
Learning Rate	3.00E-04	3.00E-04	3.00E-04	1e-3	3.00E-04
Entropy	0.01	0.01	auto_0.2	Random_action +Gaussian noise	auto_0.2
Network Size	(256 , 256)	(512,512)	(512 , 512)	(256,256,256)	(512 , 512)
Buffer Capacity	-	-	1000000	1,000,000	1000000
Batch size	4096	1024	512	256	512
Envs	16	16	8	8	32
Discount Factor	0.99	0.97	0.995	0.98	0.995
Gradient Steps / GAE Lambda	0.95	0.95	64	40	64
Success Rate (Convergence)	0	Low (2 - 3 %)	80%	0	42%
Max Success Achieved	0	Low (5 - 6 %)	84%	0	70%
GPU	RTX 3060	RTX 3060	RTX 3060	RTX 3060	RTX 3060
Issue	No Success Seen	Grasp and idle - Reward Hack	No domain randomisation	Becomes idle after some steps	

Graphs

1 . SAC - No Domain Randomisation



1 . Domain Randomisation



Weekly Progress

Week 1 and 2

At this stage we were new to reinforcement learning . Our main focus was to learn the basics of reinforcement learning. We started covering basics with David Silvers's introduction to reinforcement learning. While learning we also implemented the learnings, we used Gymnasium Farama toy environments to practice. On the project side we spawned the hand in mujoco. To hold the hand we started the design of the stand and got familiar with the design of the hand and its working. We started manual operation by hand using a python script.

Week 3 and 4

Having covered the basics of reinforcement learning , we focused on implementation of learnings and studying more advanced techniques such as policy gradient methods, and Deep Q Network (DQN) . We implemented these techniques on toy environments . At this stage we built the environment set up to train the hand for dextrous manipulation .We worked on various techniques for real world detection of cube these include Open CV , Convolutional Neural Networks , Optical Character Recognition and April Tags . We shortlisted and attempted 2 main techniques side by side , i.e. April tags and CNN model , comparing their results we decided to move forward and use April tags for our main goal . In order to get familiar with SIm2Real we worked on a minor task. We played Rock Paper Scissors on an amazing hand both in reality and in simulation .

Week 5 and 6

We resolved minor works pending in our mini task of rock paper scissor task. We shifted our focus from the learning side towards our project aim . We researched and read through many Github repositories and Research papers on or related to dextrous manipulation. We started training our hands in simulation , we used different algorithms PPO , SAC , PPO + Curriculum and DDPG+HER , as explained above to train the hand . We achieved a maximum accuracy of about 26% during this stage .

Week 7 and 8

We worked rigorously to improve our accuracy in simulation training. We achieved a maximum accuracy of about 80 % in simulation with domain randomization turned off. We achieved an accuracy of 70% with domain randomisation enabled. We tried our best to deploy the trained model on real hardware.

Resources and References

Lectures and Theory

1. https://youtu.be/2pWv7GOvuf0?si=W_IKr3VwazisAVYt
2. <https://www.udacity.com/course/deep-learning-pytorch--ud188>
3. <https://spinningup.openai.com/en/latest/>

Practice

1. <https://gymnasium.farama.org/index.html>
2. <https://github.com/udacity/deep-learning-v2-pytorch>

Papers

1. <https://arxiv.org/pdf/1808.00177>
2. <https://arxiv.org/abs/2607.03570>
3. <https://arxiv.org/pdf/2003.04960>

Github Repositories

1. <https://github.com/rhoban/onshape-to-robot>
2. <https://github.com/Betatester777/AmazingHandControl>
3. https://github.com/mani-skill/ManiSkill/blob/main/mani_skill%2Fenvs%2Ftasks%2Fdexterity%2Frotate_single_object_in_hand.py
4. https://github.com/wuji-technology/wuji-mjlab/tree/main/src/wuji_mjlab/tasks/reorient